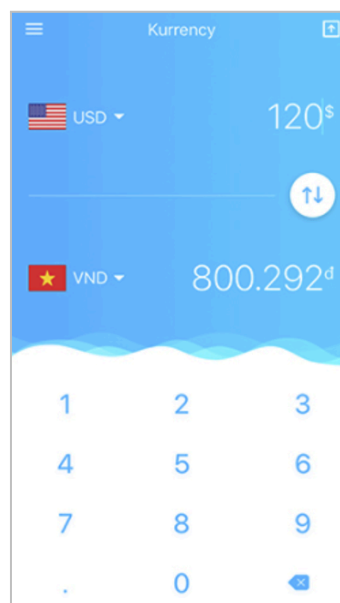


C4 Model Practice 2: Currency Exchangedawd

We are reverse-engineering the structural blueprints for **Kurrency**, the currency conversion mobile application shown in your exam prompt.

The Scenario: Currency Exchange Mobile Application



Assume that the exchange rate can be retrieved from currency web service (online). The application converts the money by submitting the amount of money and currency from both sides. At a glance, an interactive custom keypad, two currency drop-downs (USD and VND), a swap toggle, and real-time converted fields (800,292^d).

Let's look at the complete, multi-tiered **C4 Model** design for this platform.

● Level 1: System Context Diagram

Objective: Visualize the boundary of the **Kurrency System**, identifying how the human user and external financial dependencies interact with it at a macro scale.

Step-by-Step Task Instructions

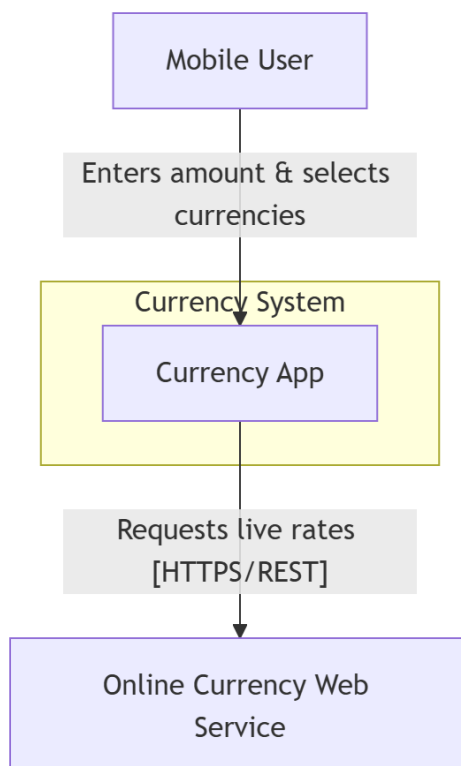
1. **Identify the Actor:** The smartphone user inputting currency figures.
2. **Define the External System:** The third-party Online Currency Web Service (e.g., Fixer.io, Open Exchange Rates).
3. **Draw Boundaries:** Represent the Kurrency ecosystem as a single system block, ignoring internal mechanics for now.

Solution Diagram

Mermaid.js

```
graph TD
    user["Mobile User"]
    subgraph currency ["Currency System"]
        app["Currency App"]
    end
    ext["Online Currency Web Service"]
    user -- "Enters amount & selects currencies" --> app
    app -- "Requests live rates [HTTPS/REST]" --> ext
```

Diagram



● Level 2: Container Diagram

Objective: Zoom inside the system boundary.

⚠ **Professor's Engineering Warning:** Do *not* connect the Mobile App directly to the external third-party API in a production system. If you embed private API access keys directly within a client binary, malicious actors will decompile the application and steal your billing tokens within minutes. We must decouple this with a secure Backend API container and an In-Memory Cache to store rates locally for up to an hour, keeping external API consumption costs low.

Step-by-Step Task Instructions

1. **Identify Frontend Containers:** Create a dedicated **Mobile Client Application**.
2. **Identify Backend Containers:** Create an intermediate **Backend API Gateway** to act as a secure proxy.
3. **Incorporate Data Storage:** Introduce an **In-Memory Rate Cache** container to manage high-speed key-value lookups.

Solution Diagram

Mermaid.js

```
classDiagram
class user{
}
namespace openExchangeRate{
    class openExchangeRateDB{
    }
    class openExchangeRateAPI{
    }
```

```

}
namespace kurrency{

    class mobileWebApp{

    }

    class backEndAPI{

    }

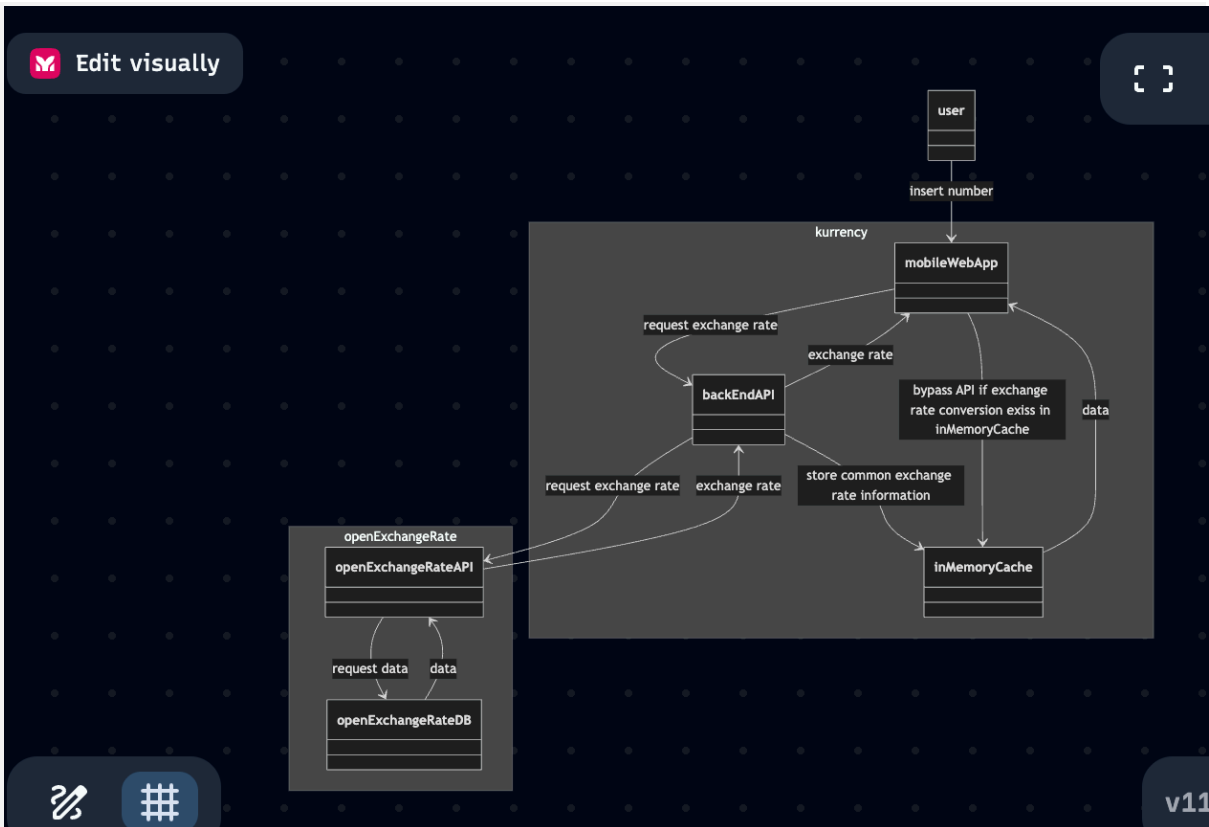
    class inMemoryCache{

    }

}

user --> mobileWebApp: insert number
mobileWebApp --> backEndAPI : request exchange rate
backEndAPI --> openExchangeRateAPI : request exchange rate
openExchangeRateAPI --> backEndAPI : exchange rate
backEndAPI --> mobileWebApp : exchange rate
backEndAPI --> inMemoryCache : store common exchange rate information
mobileWebApp --> inMemoryCache : bypass API if exchange rate conversion exists in
inMemoryCache
inMemoryCache --> mobileWebApp : data
openExchangeRateAPI --> openExchangeRateDB : request data
openExchangeRateDB --> openExchangeRateAPI : data

```



Diagram

● Level 3: Component Diagrams

Let's zoom directly into our containers to expose their modular structural blocks.

Component Diagram: 1. Kurrency Mobile App Container Components

Objective: Decompose the client-side engine running on the smartphone to see how user clicks translate to network triggers.

Step-by-Step Task Instructions

1. **Isolate View Components:** Break down the UI elements matching the prompt picture (Keypad, Dropdown Selector, Display Fields).
2. **Design the Core Controller:** Place a calculation/state coordination engine in the center to manage dynamic calculation states.
3. **Isolate the Infrastructure Client:** Establish an independent HTTP engine to make external queries.

Solution Diagram

Mermaid.js

```
classDiagram
class user{

}
namespace mobileWebApp{
```

```

class KeypadView{
}

class CurrencyDropdownSelector{
}

class ConvertedAmountDisplay{
}

class ConversionController{
}

class HTTPClient{
}

class backEndAPI{
}

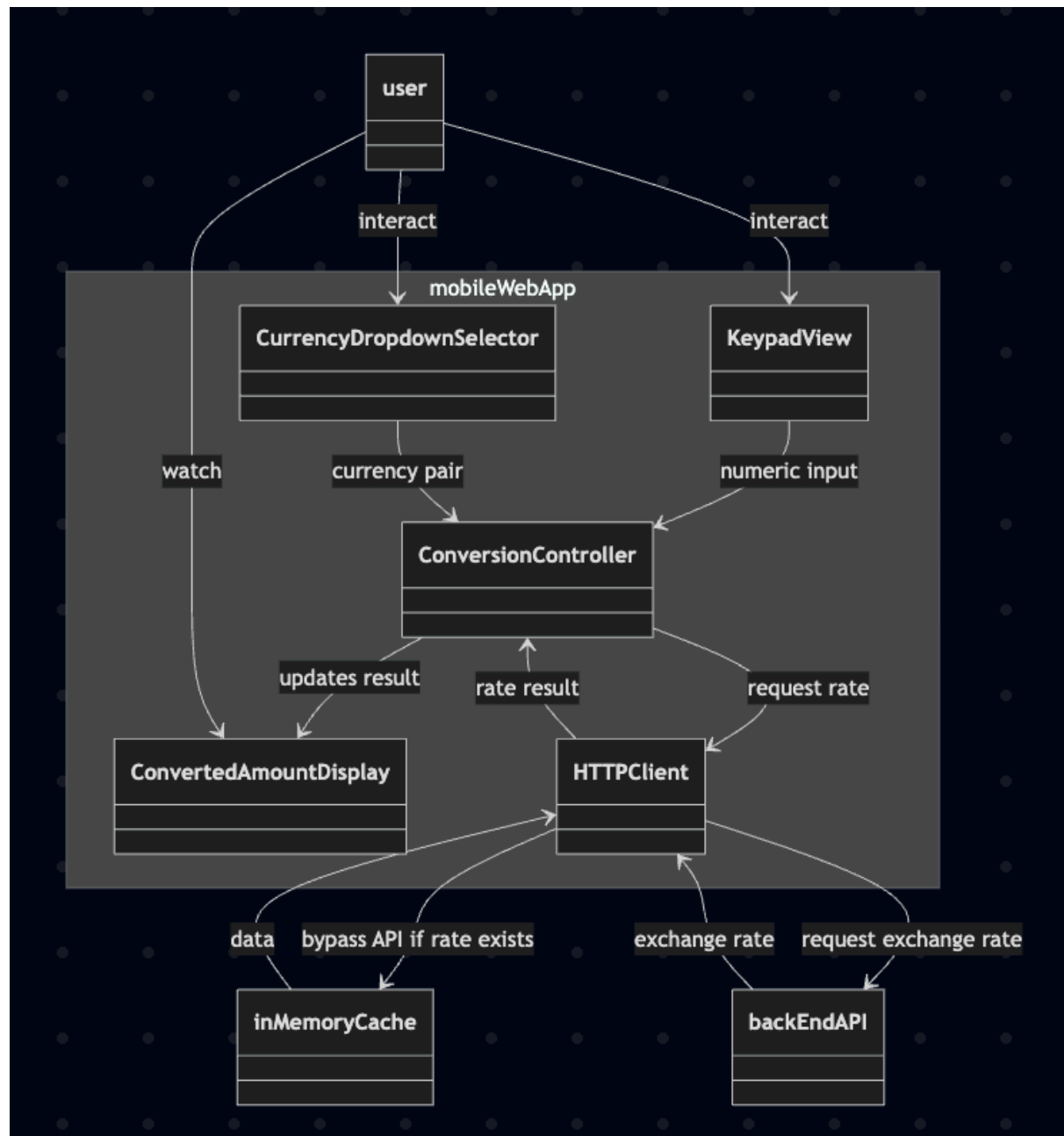
class inMemoryCache{
}

KeypadView --> ConversionController : numeric input
CurrencyDropdownSelector --> ConversionController : currency pair
ConversionController --> ConvertedAmountDisplay : updates result

```

```
ConversionController --> HTTPClient : request_rate  
  
HTTPClient --> inMemoryCache : bypass API if rate exists  
  
inMemoryCache --> HTTPClient : data  
  
HTTPClient --> backEndAPI : request_exchange_rate  
  
backEndAPI --> HTTPClient : exchange_rate  
  
HTTPClient --> ConversionController : rate_result  
  
user --> KeypadView : interact  
  
user --> CurrencyDropDownSelector : interact  
  
user --> ConvertedAmountDisplay : watch
```

Diagram



Component Diagram: 2. Secure API Backend Gateway Container Components

Objective: Decompose the backend proxy server to understand how requests are validated, processed, and optimized.

Step-by-Step Task Instructions

1. **Map Inbound Endpoints:** Define the controller listening to incoming traffic frames.
2. **Build the Integration Engine:** Isolate the logical component whose sole task is fetching and mapping raw currency matrix schemas.
3. **Connect Storage Accessors:** Link the logic handlers to the local memory cache client driver.

Solution Diagram

Mermaid.js

```
C4Component
title Component diagram - Secure API backend gateway (kurrency)

Person_Ext(user, "User", "Initiates currency conversion")

Container_Boundary(kurrency, "kurrency namespace") {
    Container(mobileWebApp, "Mobile web app", "Frontend", "User interface for currency input")

    Container_Boundary(backEndAPI, "backEndAPI - Secure gateway container") {
        Component(apiController, "API controller", "Request handler", "Validates, authenticates & routes inbound requests")
        Component(integrationService, "Exchange rate integration service", "Business logic", "Fetches & maps raw currency matrix schema from upstream provider")
        Component(cacheClientDriver, "Cache client driver", "Storage accessor", "Reads & writes exchange rate data to in-memory cache")
    }

    ContainerDb(inMemoryCache, "In-memory cache", "Cache store", "Stores common exchange rate data")
}
```

```

}

```

```

System_Ext(openExchangeRateAPI, "openExchangeRateAPI", "External exchange rate
provider")

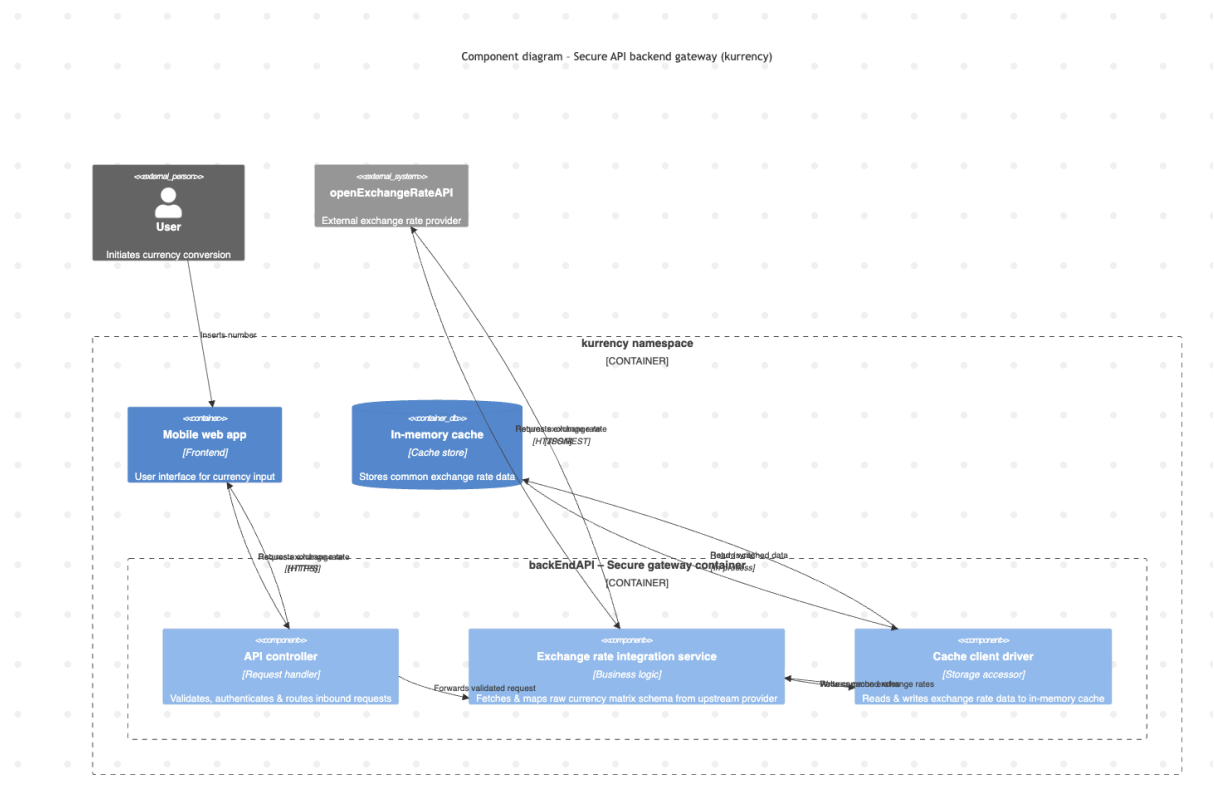
```

```

Rel(user, mobileWebApp, "Inserts number")
Rel(mobileWebApp, apiController, "Requests exchange rate", "HTTPS")
Rel(apiController, integrationService, "Forwards validated request")
Rel(integrationService, openExchangeRateAPI, "Requests exchange rate",
"HTTPS/REST")
Rel(openExchangeRateAPI, integrationService, "Returns exchange rate", "JSON")
Rel(integrationService, cacheClientDriver, "Write common exchange rates")
Rel(cacheClientDriver, inMemoryCache, "Read / write", "In-process")
Rel(inMemoryCache, cacheClientDriver, "Returns cached data")
Rel(cacheClientDriver, integrationService, "Returns cached rates")
Rel(apiController, mobileWebApp, "Returns exchange rate", "HTTPS")

```

Diagram



X